

Let's get into action with Playwright

Agenda – Part 1 (Playwright Basics)

- Introduction to Playwright
- Selenium Architecture vs Playwright
- Playwright documentation

The Golden Circle

What

What is Playwright

A **modern automation** tool for web browsers that allows you to automate tasks, interact with web pages, and perform end-to-end testing across multiple browsers and platforms.

Why

Why Playwright?

Playwright supports automatic waiting for **page elements, device emulation and the ability to record and capture screenshots and videos during testing.**

How

How is PW better than others?

Playwright surpasses other automation tools with **its cross-browser and cross- platform compatibility, faster execution, robust parallelization, and comprehensive capabilities** for automating web interactions and testing.

Intro to Playwright

- A modern, open source web test framework from Microsoft
- Manipulates the browser via (superfast) debug protocols
- Works with Chromium/Chrome/Edge, Firefox, & WebKit
- Provides automatic waiting, test generation, UI mode, etc.
- Can test UIs and APIs together
- Bindings for JavaScript, Python, Java, & C#
 - TypeScript is recommended

Tool – Scripting Languages



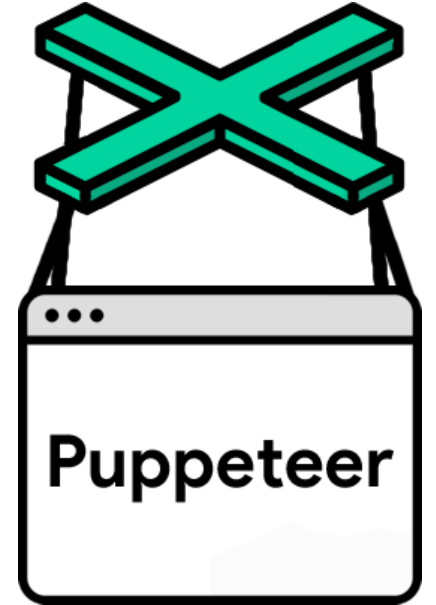
Selenium



Playwright



Cypress



Puppeteer

Core Java,
Python,
C#,
JavaScript,
Ruby,
Perl

Core Java,
Python,
C#,
JavaScript

JavaScript

JavaScript

WebSocket Protocol working alongside DevTools Protocol

What is WebSocket?

The **WebSocket protocol** is a **communication protocol** that provides **full-duplex** (two-way) communication channels over a **single, long-lived connection** with a stateful / persistent connection style.

Unlike **HTTP protocol**, which is request-response based, WebSockets allow for continuous, real-time communication between a **client** (like a web browser) and a **server**, with stateless/ short lived connection style.

Where it is used :

It allows servers to **push** data to clients **without** the client having to request it, which is ideal for **real-time applications** like:

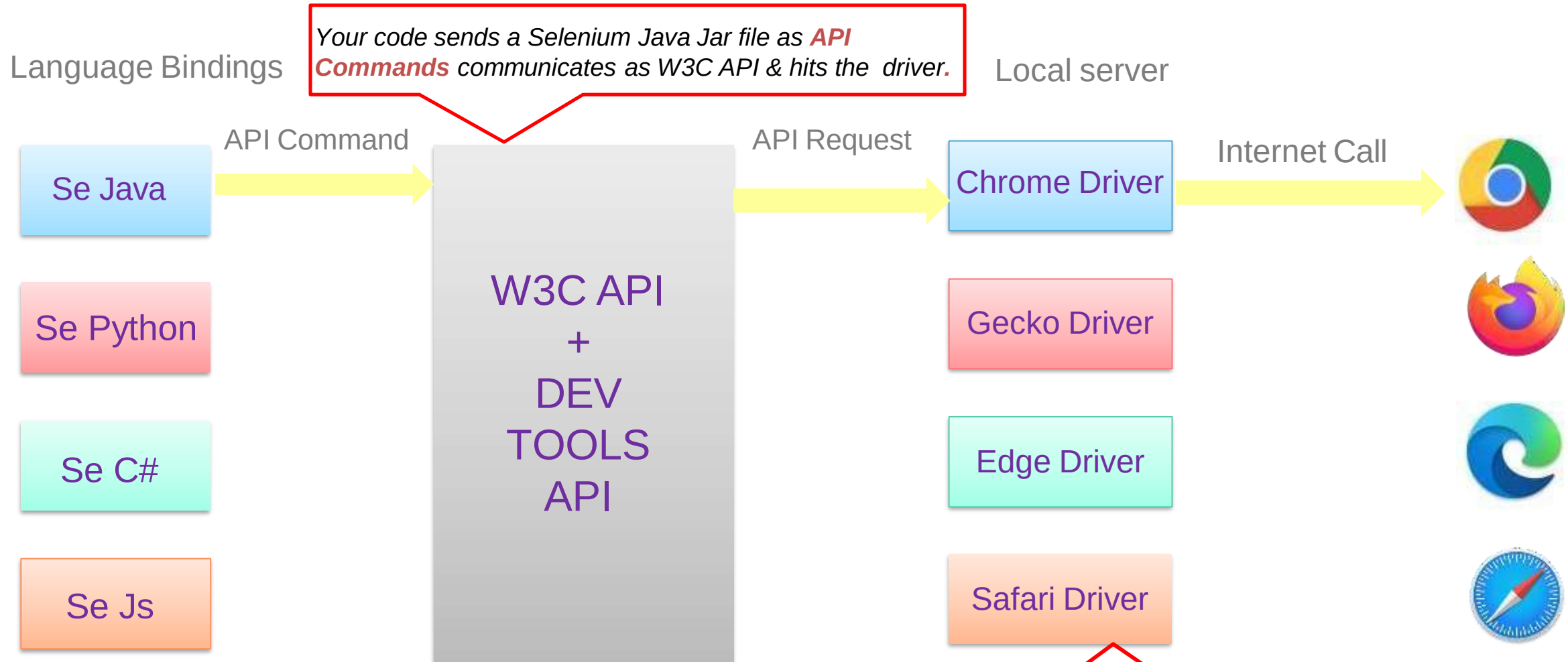
- Chat applications
- Online gaming
- Live financial dashboards

How it works :

The WebSocket connection communicates directly with **Chrome's DevTools Protocol (CDP)** to:

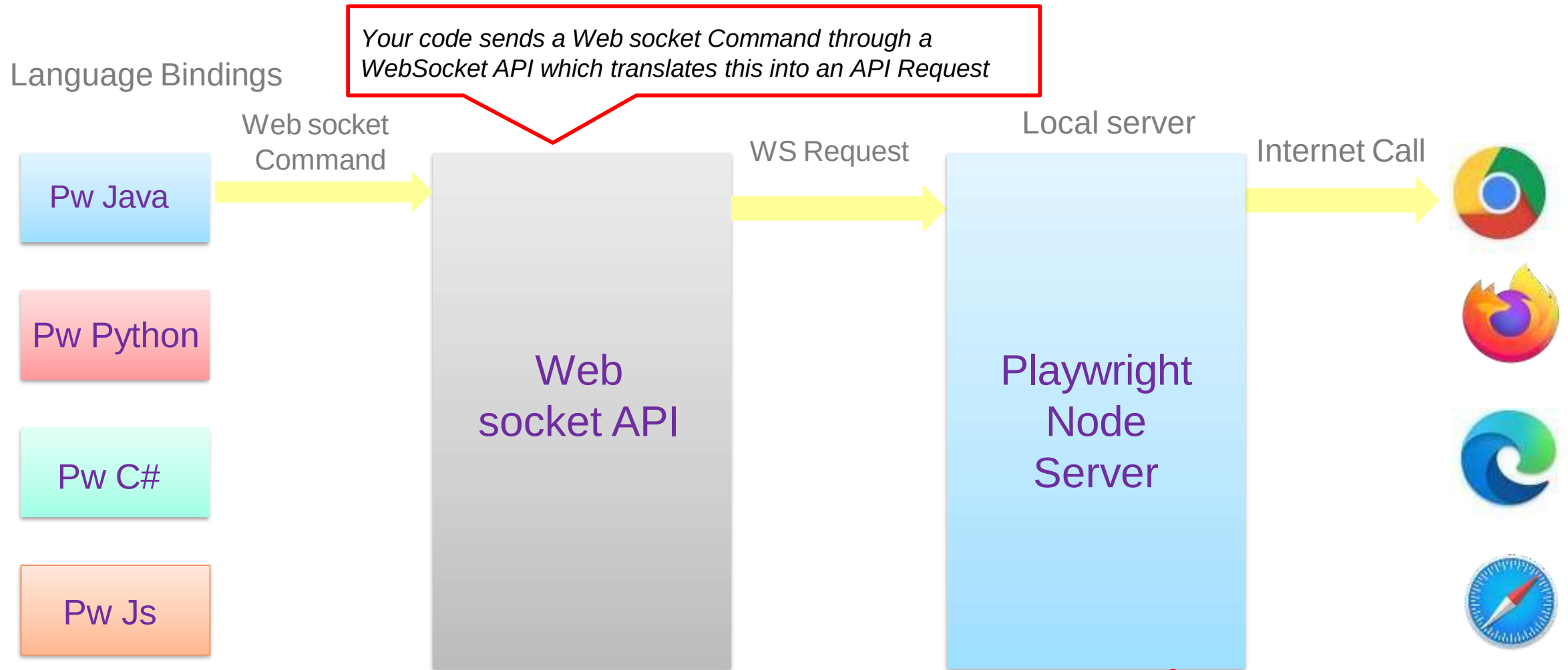
- Capture **network traffic**.
- Inspect **DOM elements**.
- Perform **performance audits**.

Selenium Architecture



Once the driver receives the **API commands**, it communicates to the browser via an **Internet Call**.
(Usually in Python code)

Playwright Architecture

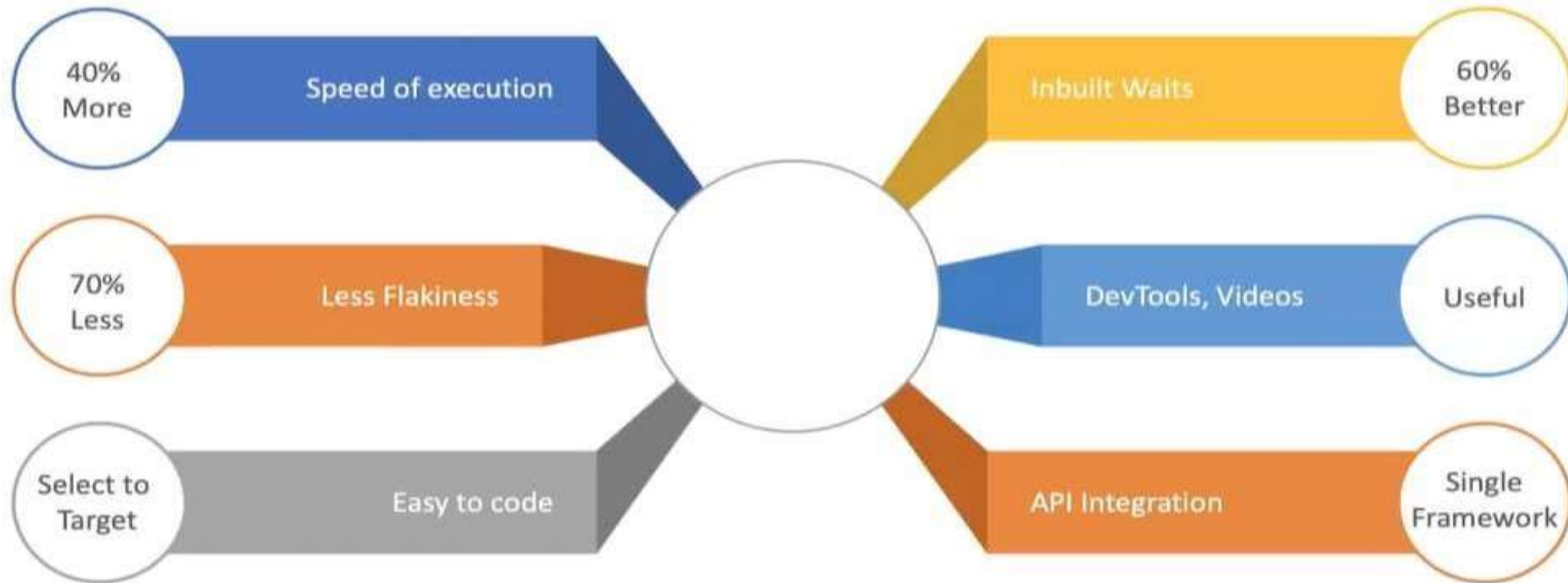


The Playwright Node Server sends commands to the browser devtools. The browser executes the actions and communicates back through the same WebSocket channel.

Selenium Vs Playwright Architecture Difference

Feature	Selenium	Playwright
Protocol	W3C WebDriver Protocol	WebSocket API
Drivers	Requires browser-specific drivers (e.g., ChromeDriver)	No drivers needed; uses a single Node server
Speed	Slower due to extra layers of communication	Faster due to WebSocket's direct communication
Setup Complexity	Higher (driver installation needed)	Easier (drivers are bundled with Playwright)
Parallel Testing	Limited without additional setup	Built-in support for parallel testing

WHY PLAYWRIGHT?



Playwright Documentations



Playwright

Docs

API

1.22 ▾

Node.js ▾



Search  

Playwright enables reliable end-to-end testing for modern web apps.

GET STARTED

 Star

39k+



Any browser • Any platform • One API

Link to doc: <https://playwright.dev/>

Agenda – Part 2 (Playwright Key Concepts)

- Browser
- BrowserContext
- Page
- DOM
- WebElement

Browser, Browser Context & Page

Browser

SESSION ISOLATION

SEPARATION OF STATE



Amazon



Flipkart



Gmail



SECURITY

Username:

Password:

Login

Username:

Password:

Login

Username:

Password:

Login

INDEPENDENT EXECUTION

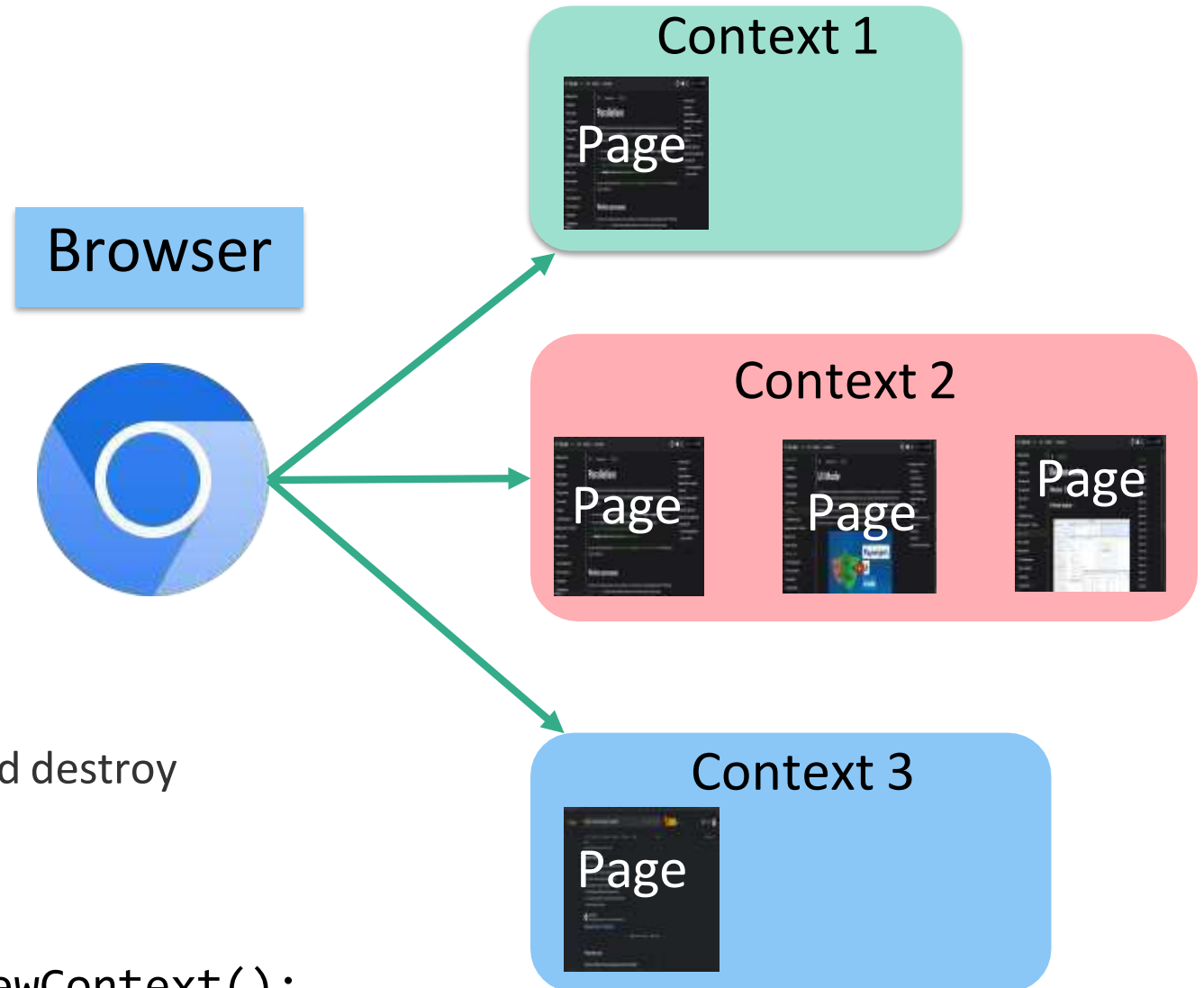
Pages

Browser Context – Isolated Environment

Browser & Browser Context

Context

- Creates a ***unique browser context*** from that instance for each test.
- A browser context is essentially like an ***incognito session***: it has its own session storage and tabs that are not shared with any other context.
- Browser contexts are very fast to create and destroy
- Code Snippet
`const context = await browser.newContext();`



Browser Context & Pages

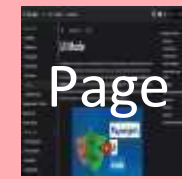
Page

- Each browser context can have one or more *pages*.
- All Playwright interactions happen through a page, like clicks and scrolls.
- Most tests only ever need one page.
- Code Snippet
`const page = await context.newPage();`

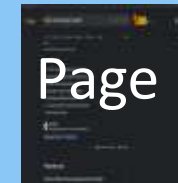
Context 1



Context 2



Context 3



Implementation of Browser Context and Page

```
import { chromium, test } from "@playwright/test";

test(`Test to launch the browser`, async () => {

    // Create a browser instance
    const browser = await chromium.launch({headless: false, channel: 'chrome'});

    // Create the browser context
    const context = await browser.newContext();

    // Create a new page
    const page = await context.newPage();

    // Load the url
    await page.goto("http://leaftaps.com/opentaps/control/main");

    // Get the title of the page
    console.log(`The title of the page is ${ await page.title()}`);
    await page.waitForTimeout(5000);

})
```

Implementation of Browser Context and Page

Line 1

```
import { chromium, test } from "@playwright/test";
```

- `chromium` : Imports the Chromium browser engine from Playwright.
- `test` : Imports the test function from Playwright's test framework, which is used to define test cases.

Line 2

Test title as string.

Asynchronous anonymous arrow function as callback function.

```
test(`Test to launch the browser`, async () => {
```

- “**test**” is a function in Playwright that allows you to define test cases with a title and an asynchronous test body.
- Since Playwright interactions return Promises, “**async**” enables us to wait for actions like page navigation.

Playwright Test execution – Analogy to RELAY RACE.

❑ Organizer Starts the Race (Test Runner Initiates the Test):

The **race organizer** starts the event.

Similarly, when you run ``npx playwright test``, the **Playwright Test runner** starts looking for test cases (``test()`` functions) to execute.

❑ First Runner (Launch Browser):

The **first runner** holds the baton and starts running. In your test, this is like **launching the browser**:

```
const browser = await chromium.launch({headless: false, channel: 'chrome'});
```

❑ Second Runner (Create Context & New Page):

The second runner grabs the baton only when the first runner reaches them. This is like creating the browser context and opening a new page:

```
const context = await browser.newContext();
```

```
const page = await context.newPage();
```



Note : Without waiting, the test could fail—just like dropping the baton in a relay race!

Playwright Test execution – Analogy to RELAY RACE.

❑ Third Runner (Navigate to URL):

The **third runner** runs after getting the baton. This corresponds to **navigating to the URL**:

```
// Load the url  
await page.goto("http://leaftaps.com/opentaps/control/main");
```

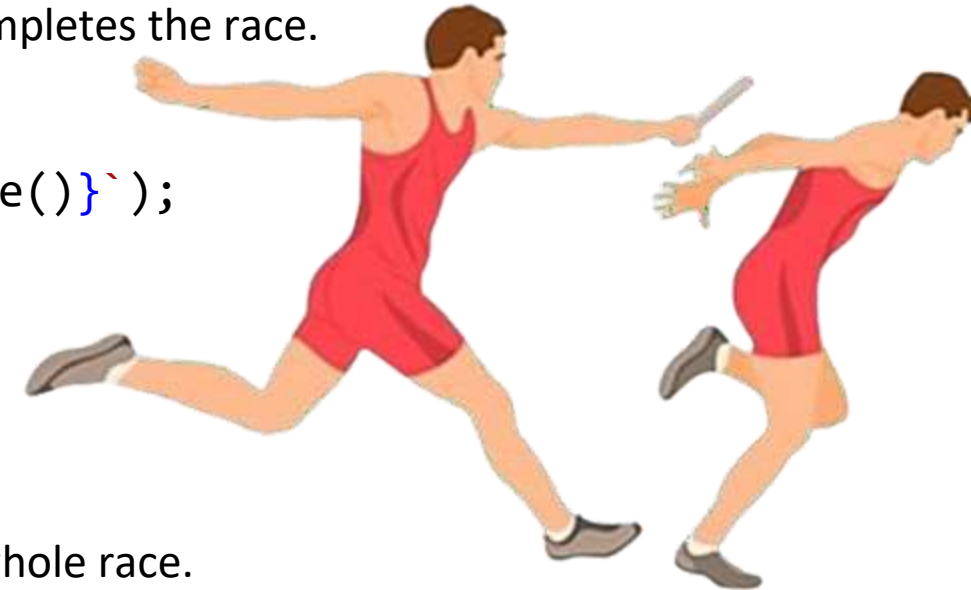
❑ Fourth Runner (Get Title & Finish the Race):

The **final runner** grabs the baton, sprints to the finish line, and the team completes the race. This is like **fetching the page title** and **waiting for a few seconds**:

```
console.log(`The title of the page is ${ await page.title()}`);  
await page.waitForTimeout(5000);
```

❑ Why `async/await` is Like a Smooth Baton Pass

- In a relay race, if one runner starts too soon or too late, it messes up the whole race.
- In Playwright, if you don't use `await`, the next step might start before the previous one is done, causing errors (like trying to get a page title before the page is even loaded!).



Playwright *config* file walkthrough

- *testDir* Contains dir path where all test cases resides
- *fullyParallel* Decides if to run all files in parallel or all. Files and test cases inside them in parallel
- *Test timeout* Max time allowed for a test case to run
- *retries* Number of time a test case to retry on failure
- *workers* Number of workers to split the test cases on
- *trace* On what conditions, we should keep the trace
- *screenshots* On what conditions, we should keep the screenshot
- *video* On what conditions, we should keep the video
- *Expect timeout* Max time allowed for assertions we are doing using expect

Playwright *config* file walkthrough

Understanding Three Levels of Configuration in Playwright

When running Playwright tests (`.spec.ts` files), there are three levels of configurations that determine how the tests execute. The priority order is as follows:

1. CLI (Command Line Interface) Options → Highest Priority
2. Test-Level Configuration (Within test script) → Higher than config file
3. `playwright.config.ts` (Global configuration file) → Lowest Priority

Key Differences in CLI Execution

Feature	Using <code>page</code> Fixture	Manually Creating Browser/Context/Page
Browser Creation	Managed by Playwright	Needs to be manually launched
Context per test	New context for each test	Context persists across tests
Parallel Execution	Default parallel tests	Must manage manually
CLI Options Applied?	✅ Yes, CLI overrides <code>playwright.config.ts</code>	❌ No, must be coded manually
Best For	Simple tests, CLI overrides, parallel execution	Full browser control, persistent sessions